



Spark Real time

Python

Apache Spark è un framework open source pensato per elaborare grandi quantità di dati in modo veloce, distribuito e scalabile.

In parole semplici, Spark permette di prendere dati molto grandi, dividerli tra più computer o nodi di un cluster, elaborarli in parallelo e ottenere risultati in tempi molto più rapidi rispetto a un'elaborazione tradizionale su una sola macchina.



Spark nasce nel mondo Big Data come alternativa più moderna e flessibile rispetto a strumenti come Hadoop MapReduce.

La sua forza principale è la capacità di lavorare molto in memoria, cioè mantenendo i dati temporanei nella RAM quando possibile.

Questo riduce la necessità di leggere e scrivere continuamente su disco, rendendo molte operazioni più veloci, soprattutto quando si devono fare analisi ripetute, trasformazioni complesse o calcoli su dataset molto grandi.



Lavorare con dati real time significa gestire dati che vengono prodotti, ricevuti ed elaborati mentre stanno accadendo, senza aspettare di raccogliarli tutti prima di analizzarli.

In un'elaborazione tradizionale, detta spesso batch, i dati vengono prima accumulati e poi analizzati in un secondo momento.

Per esempio: un'azienda raccoglie tutte le vendite della giornata e la sera genera un report.



In questo caso il dato non viene usato subito, ma solo dopo un certo intervallo di tempo.

Nel real time, invece, il sistema prova a reagire quasi immediatamente all'arrivo del dato.

Per esempio, quando un pagamento con carta viene controllato per capire se è sospetto, quando una dashboard mostra in tempo reale gli accessi a un sito, quando un sensore industriale segnala un'anomalia o quando un'app di delivery aggiorna la posizione del rider sulla mappa.

I dati non arrivano come un file unico già completo, ma come eventi continui: una transazione, un click, una posizione GPS, un messaggio, una temperatura rilevata da un sensore, un log generato da un server.



Nel contesto di Spark, lavorare con dati real time significa usare Spark per leggere questi flussi continui, trasformarli, filtrarli, aggregarli e produrre risultati aggiornati man mano.

Spark, in particolare con strumenti come Spark Structured Streaming, permette di trattare i dati in arrivo come se fossero una tabella che si aggiorna continuamente.



È importante però distinguere tra real time puro e near real time.

Il real time puro richiede risposte quasi istantanee, spesso nell'ordine dei millisecondi.

Il near real time accetta piccoli ritardi, per esempio pochi secondi o qualche decina di secondi.

Spark viene spesso usato in scenari near real time, dove non serve reagire al millisecondo, ma è comunque necessario avere risultati molto aggiornati.



- **readStream**

Serve per leggere dati in modalità streaming. A differenza di una normale lettura batch, dove Spark legge un file o una tabella già completa, readStream indica a Spark che i dati arriveranno in modo continuo da una sorgente.

Per esempio, può leggere eventi da Kafka, file che vengono aggiunti progressivamente in una cartella, dati da socket o sorgenti cloud.

È il punto di partenza di un'applicazione real time in Spark.



- **writeStream**

**Serve per scrivere il risultato di uno stream.
Dopo aver letto e trasformato i dati real time, Spark deve inviarli verso una destinazione finale.**

Questa destinazione può essere la console, una tabella, un file, un database o un altro sistema di analisi.

writeStream definisce quindi dove devono finire i dati elaborati e in che modo devono essere aggiornati.



- **select**

Serve per scegliere quali colonne mantenere o trasformare.

Nei dati real time spesso arrivano molte informazioni insieme, ma non tutte sono realmente utili per l'analisi.

Con select si possono estrarre solo i campi necessari, come ID utente, timestamp, valore della transazione, messaggio ricevuto o codice errore.



È utile anche per riorganizzare o preparare i dati prima delle elaborazioni successive.

- **filter / where**

Servono per filtrare i dati in ingresso. In uno stream continuo non sempre interessa analizzare tutto ciò che arriva: spesso si vogliono mantenere solo gli eventi rilevanti.

Per esempio, si possono selezionare solo transazioni sopra una certa cifra, solo log con errori, solo sensori con valori anomali o solo eventi provenienti da una specifica area.



filter e where hanno un ruolo simile e permettono di ridurre il flusso ai dati davvero importanti.

- **withColumn**

Serve per aggiungere una nuova colonna o modificare una colonna esistente.

In uno scenario real time può essere usato per calcolare nuovi valori, convertire formati, aggiungere categorie o normalizzare dati ricevuti in forma grezza.

Per esempio, si può trasformare una stringa in timestamp, aggiungere una colonna che indica se una transazione è sospetta, oppure calcolare una fascia di rischio partendo da un valore numerico.



- **groupBy**

Serve per raggruppare i dati secondo uno o più criteri.

In ambito real time è utile quando vogliamo osservare non il singolo evento, ma un insieme di eventi con una caratteristica comune.

Per esempio, possiamo raggruppare gli accessi per utente, le vendite per prodotto, gli errori per tipo, oppure i dati dei sensori per macchina.



Spesso viene usato insieme a funzioni di aggregazione come conteggio, somma, media, massimo o minimo.

- **window**

Serve per dividere il flusso di dati in intervalli temporali. È una funzione fondamentale nello streaming perché un flusso real time, teoricamente, non finisce mai.

Per poterlo analizzare, bisogna quindi ragionare su finestre di tempo: per esempio, contare gli accessi ogni 5 minuti, calcolare le vendite ogni ora o monitorare la media della temperatura negli ultimi 10 minuti. window permette di dare una struttura temporale ai dati continui.



- **start**

Serve per avviare concretamente il processo di streaming.

Prima di start, Spark ha solo definito le operazioni da eseguire: lettura, trasformazioni, filtri, aggregazioni e scrittura.

Con start, invece, l'applicazione comincia davvero a leggere i dati dalla sorgente, elaborarli e produrre risultati aggiornati.

È il metodo che fa partire l'elaborazione continua dello stream



Esempio 1: leggere dati real time da socket e stamparli a console

```
1.from pyspark.sql import SparkSession
2.
3.# Creo la sessione Spark
4.spark = SparkSession.builder \
5.  .appName("EsempioStreamingSocket") \
6.  .getOrCreate()
7.
8.# Leggo dati in streaming da un socket
9.dati = spark.readStream \
10.  .format("socket") \
11.  .option("host", "localhost") \
12.  .option("port", 9999) \
13.  .load()
14.
15.# Scrivo i dati ricevuti sulla console
16.query = dati.writeStream \
17.  .format("console") \
18.  .start()
19.
20.# Mantengo attivo lo streaming
21.query.awaitTermination()
```

Questo esempio mostra il caso più semplice possibile: Spark si collega a una sorgente che invia dati in tempo reale, in questo caso un socket sulla porta 9999, e stampa ogni dato ricevuto sulla console.

È utile per far capire che uno stream non lavora su un file già completo, ma su dati che continuano ad arrivare mentre il programma è in esecuzione.



Esempio 2: filtrare eventi real time

```
1. from pyspark.sql import SparkSession
2.
3. spark = SparkSession.builder \
4.     .appName("FiltroStreaming") \
5.     .getOrCreate()
6.
7. # Lettura dello stream
8. dati = spark.readStream \
9.     .format("socket") \
10.    .option("host", "localhost") \
11.    .option("port", 9999) \
12.    .load()
13.
14. # Filtro solo le righe con la parola ERRORE
15. errori = dati.filter(dati.value.contains("ERRORE"))
16.
17. # Scrittura del risultato in console
18. query = errori.writeStream \
19.    .format("console") \
20.    .start()
21.
22. query.awaitTermination()
```

In questo esempio Spark riceve dati in tempo reale e mantiene solo quelli che contengono la parola **ERRORE.**

È un caso molto vicino al monitoraggio dei log: invece di leggere tutto ciò che arriva, il sistema seleziona solo gli eventi rilevanti.

Questo è uno degli usi più comuni dello streaming, perché nei flussi real time spesso non interessa tutto il dato grezzo, ma solo ciò che richiede attenzione.



Esempio 3: aggiungere una colonna con withColumn

```
1.from pyspark.sql import SparkSession
2.from pyspark.sql.functions import
  current_timestamp
3.
4.spark = SparkSession.builder \
5.  .appName("WithColumnStreaming") \
6.  .getOrCreate()
7.
8.# Lettura dello stream
9.dati = spark.readStream \
10.  .format("socket") \
11.  .option("host", "localhost") \
12.  .option("port", 9999) \
13.  .load()
14.
15.# Aggiungo una colonna con il timestamp
16.dati_con_orario =
  dati.withColumn("orario_elaborazione",
    current_timestamp())
17.
18.# Scrittura in console
19.query = dati_con_orario.writeStream \
20.  .format("console") \
21.  .start()
22.
23.query.awaitTermination()
```

Qui Spark riceve dati in tempo reale e aggiunge una nuova colonna chiamata orario_elaborazione. Questa colonna contiene il momento in cui Spark ha elaborato il dato.

È un esempio utile per spiegare che nello streaming spesso non basta ricevere il dato, ma bisogna anche arricchirlo con informazioni aggiuntive, come orari, categorie, stati o valori calcolati.



Esempio 4: contare eventi per finestra temporale

```
1.from pyspark.sql import SparkSession
2.from pyspark.sql.functions import current_timestamp,
  window
3.
4.spark = SparkSession.builder \
5.  .appName("WindowStreaming") \
6.  .getOrCreate()
7.
8.# Lettura dello stream
9.dati = spark.readStream \
10.  .format("socket") \
11.  .option("host", "localhost") \
12.  .option("port", 9999) \
13.  .load()
14.
15.# Aggiungo un timestamp ai dati ricevuti
16.dati_con_orario = dati.withColumn("orario",
  current_timestamp())
17.
18.# Conto quanti eventi arrivano ogni 10 secondi
19.conteggio = dati_con_orario.groupBy(
20.  window(dati_con_orario.orario, "10 seconds")
21.  ).count()
22.
23.# Scrittura in console
24.query = conteggio.writeStream \
25.  .format("console") \
26.  .outputMode("complete") \
27.  .start()
28.
29.query.awaitTermination()
```

Questo esempio introduce il concetto di finestra temporale.

Poiché uno stream non ha una fine naturale, Spark deve ragionare per intervalli di tempo. In questo caso conta quanti eventi arrivano ogni 10 secondi.

È un esempio fondamentale per spiegare dashboard real time, monitoraggio accessi, conteggio errori, analisi sensori o controllo delle transazioni in intervalli brevi.



In conclusione, Spark è uno strumento molto potente per lavorare con grandi quantità di dati, ma diventa particolarmente interessante quando viene usato in scenari real time o near real time.

In questi contesti, il valore non sta solo nel raccogliere i dati, ma nel riuscire ad analizzarli mentre arrivano, trasformandoli rapidamente in informazioni utili.

Con Spark Structured Streaming è possibile leggere flussi continui di dati, filtrarli, trasformarli, arricchirli, raggrupparli e analizzarli per finestre temporali, questo permette di costruire sistemi capaci di monitorare , rilevare anomalie, aggiornare dashboard, controllare transazioni o analizzare log quasi in tempo reale.



Buon Davante a tutti

